GLOBAL DISASTER MANAGEMENT SYSTEM

Minor Project Report

Submitted By

Raghavendra N 4NI24IS147
Mohit S 4NI24IS108
Shreeram S 4NI24IS187
N Advik 4NI24IS111
Pravesh KJ 4NI24IS143

ABSTRACT

The Global Disaster Management System is a full-stack disaster intelligence and monitoring platform developed using React, Node.js, Express, Supabase PostgreSQL, Globe.gl, and Recharts. The system integrates multiple real-world disaster datasets such as earthquakes from USGS, wildfire data from NASA FIRMS, flood-risk analysis using OpenWeather APIs, and conflict-zone data from ACLED datasets. The project provides a centralized platform for monitoring global disasters using interactive dashboards, globe-based visualization, analytical charts, and database-driven event management. The application follows a normalized database architecture with secure CRUD operations, authentication, and PostgreSQL triggers for maintaining database integrity. The platform enables administrators to create, update, delete, and monitor manually added disaster events while preserving external API-based records as read-only. Interactive globe visualization, smooth trend analysis graphs, and real-time filtering provide a modern analytical interface for understanding disaster distribution across the world. The project demonstrates the integration of database management systems, real-time APIs, visualization techniques, backend development, and cloud-hosted PostgreSQL systems into a unified disaster intelligence platform.

1. INTRODUCTION

Natural disasters and global crises have become increasingly frequent and severe in recent years. Traditional disaster monitoring systems are often fragmented and fail to provide unified visualization, centralized management, and efficient data analysis. The Global Disaster Management System addresses these challenges by integrating multiple disaster sources into a single analytical platform. The system visualizes earthquakes, floods, wildfires, and conflict-zone events on an interactive globe and analytical dashboard. The platform uses real-world datasets and cloud database systems to maintain scalability, accessibility, and efficient query execution.

1.1 Problem Statement

Existing disaster monitoring systems lack centralized visualization, efficient database management, and integrated analytical dashboards. Many systems provide isolated information for individual disaster categories without supporting comparative analysis or unified management.

1.2 Objectives

- Build a centralized disaster intelligence platform.
- Integrate multiple disaster sources such as earthquakes, floods, wildfires, and conflict zones.
- Implement secure CRUD operations using Supabase PostgreSQL.
- Provide interactive globe-based visualization and analytical charts.
- Demonstrate normalized database design and trigger-based automation.

1.3 Scope of the Project

The project focuses on disaster monitoring, database management, and analytical visualization using cloud-hosted APIs and PostgreSQL systems. The application supports global event tracking, administrative data management, and disaster trend analysis.

2. SYSTEM REQUIREMENTS

Hardware Requirements
• Intel i5 / Ryzen 5 or higher
• Minimum 8GB RAM
• Stable Internet Connection

Software Requirements
• React + Vite
• Node.js + Express.js
• Supabase PostgreSQL
• Globe.gl
• Recharts
• Tailwind CSS + shadcn/ui
• VS Code

3. SYSTEM DESIGN

The project follows a modular full-stack architecture. Frontend: • React dashboard
• Globe visualization
• Charts and analytics
Backend: • Node.js + Express API layer
• Authentication and CRUD APIs
• Data cleaning and transformation
Database: • PostgreSQL database hosted on Supabase
• Normalized schema with indexing and triggers

3.1 E-R Diagram

The database contains the following entities: • disaster_events
• event_types
• locations
• admin_users
• analytics_logs
Relationships are maintained using foreign keys to ensure normalization and reduce redundancy.

3.2 Schema Diagram

The database schema follows Third Normal Form (3NF). Event types and locations are separated into dedicated tables to avoid duplicate storage.

3.3 Data Flow Diagram

API Sources → Backend Processing → Supabase Database → Frontend Visualization External APIs: • USGS (Earthquakes)
• NASA FIRMS (Wildfires)
• OpenWeather (Flood Risk)
• ACLED (Conflict Zones)

4. DATABASE DESIGN

The database uses PostgreSQL hosted on Supabase with normalized relational tables. Key Features: • Primary Keys
• Foreign Keys
• Indexing
• Triggers
• Soft Delete Mechanism
• Row-level CRUD Operations

4.1 Table Structures

Tables: 1. disaster_events
2. event_types
3. locations
4. admin_users
5. analytics_logs

4.2 Normalization

The database follows 3NF: • Event types stored separately
• Locations normalized into dedicated table
• Foreign keys used to avoid redundancy

4.3 SQL Queries

Example Queries:

```
SELECT * FROM disaster_events;
SELECT * FROM disaster_events WHERE source_type='manual';
SELECT type, COUNT(*) FROM disaster_events GROUP BY type;
```

5. IMPLEMENTATION

Frontend: • React dashboard with dark futuristic UI
• Globe visualization using Globe.gl
• Interactive charts using Recharts

Backend: • REST APIs using Express.js
• JWT Authentication
• Supabase integration
• CSV parsing for ACLED datasets

5.1 Frontend Design

The frontend includes: • Landing page with animated globe
• Dashboard with filters and charts
• Globe page with event visualization
• Admin mode for CRUD operations

5.2 Backend Connectivity

Backend services fetch, clean, and insert disaster data into Supabase automatically. The backend also handles authentication and row-level CRUD operations.

5.3 Sample Input and Output Screenshots

The application displays: • Disaster events on globe
• Interactive charts
• Event filtering
• Admin CRUD panels

6. TESTING

Testing includes: • CRUD validation
• Authentication testing
• API integration testing
• Database trigger testing
• Visualization testing

6.1 Test Cases

• Add new disaster event
• Update manual event
• Delete manual event
• Validate admin login
• Verify globe updates

6.2 Test Results

All CRUD operations, trigger automation, and visualization modules executed successfully with proper database consistency.

CONCLUSION

The Global Disaster Management System successfully integrates multiple disaster sources into a centralized cloud-based analytical platform. The project demonstrates database normalization, secure CRUD operations, visualization systems, and API integration. The system provides scalable architecture for future expansion into

predictive analytics and AI-driven disaster forecasting.

FUTURE ENHANCEMENTS

- AI-based disaster prediction
- Real-time notification system
- Mobile application support
- Satellite image analytics
- Machine learning-based risk analysis

REFERENCES

- USGS Earthquake API
- NASA FIRMS
- OpenWeather API
- ACLED Dataset
- Supabase Documentation
- Globe.gl Documentation
- Recharts Documentation